RAPPORT TECHNIQUE - MyGES Mobile Application

Projet : Application Mobile MyGes
Framework : Jetpack Compose
Établissement : ESGI Aix en Provence
Date : Mars 2026
Élève : Rusescu Alexandru

Table des matières

1. Introduction
2. Architecture de l'Application
3. Fonctionnalités Implémentées
4. Technologies et Dépendances
5. Interface Utilisateur
6. Gestion des Données
7. Sécurité
8. Internationalisation
9. Points d'Amélioration
10. Conclusion

---

1. Introduction

1.1 Objectif du Projet

L'application **MyGES** est une application mobile Android native développée avec Jetpack Compose destinée aux étudiants du Campus Eductive d'Aix-en-Provence. Elle centralise les services essentiels du campus en intégrant l'API officielle MyGES.

1.2 Périmètre Fonctionnel

L'application permet aux étudiants de :

- Consulter l'actualité du campus
- Accéder à leur emploi du temps
- Consulter leurs notes par trimestre
- Visualiser leur profil étudiant

1.3 Contraintes Techniques

- **SDK Minimum** : Android 9.0 (API 28)
- **SDK Cible** : Android 15 (API 36)
- **Langage** : Kotlin 2.2.21
- **Framework UI** : Jetpack Compose
- **API Externe** : MyGES API (https://github.com/Melv1no/myges-api)

---

2. Architecture de l'Application

2.1 Architecture Globale

L'application suit les principes de **Clean Architecture** avec une séparation claire en trois couches :

```
┌─────────────────────────────────────────────────────┐
│                PRESENTATION LAYER (UI)              │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐   │
│   │   Screens    │   │ ViewModels│   │    Theme     │   │
│   └──────────────┘   └──────────────┘   └──────────────┘   │
└─────────────────────────────────────────────────────┘
                          ↕
┌─────────────────────────────────────────────────────┐
│                 DOMAIN LAYER                        │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐   │
│   │Repositories│   │  Entities │   │    Enums     │   │
│   │(Interfaces)│   │           │   │              │   │
│   └──────────────┘   └──────────────┘   └──────────────┘   │
└─────────────────────────────────────────────────────┘
                          ↕
┌─────────────────────────────────────────────────────┐
│                   DATA LAYER                        │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐   │
│   │Repository │   │  Network  │   │    DTOs      │   │
│   │    Impl      │   │  Services │   │              │   │
│   └──────────────┘   └──────────────┘   └──────────────┘   │
└─────────────────────────────────────────────────────┘
```

2.2 Pattern MVVM

Chaque écran suit le pattern **MVVM (Model-View-ViewModel)** :

- **View** : Composables Jetpack Compose (UI déclarative)
- **ViewModel** : Gestion de l'état et logique métier
- **Model** : Entités du domaine et données

Exemple avec l'écran Home :

```
// View (HomeScreen.kt)
@Composable
fun HomeScreen(viewModel: HomeViewModel) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
    // UI rendering based on state
}

// ViewModel (HomeViewModel.kt)
class HomeViewModel(private val repository: NewsRepository) : ViewModel()
{
    private val _uiState = MutableStateFlow<HomeUiState>(Loading)
    val uiState: StateFlow<HomeUiState> = _uiState.asStateFlow()
}

// Repository Interface (NewsRepository.kt - Domain)
interface NewsRepository {
    suspend fun getNews(): Result<List<News>>
}

// Repository Implementation (NewsRepositoryImpl.kt - Data)
class NewsRepositoryImpl(private val api: ApiService) : NewsRepository {
    override suspend fun getNews() = // API call
}
```

2.3 Navigation

L'application utilise une architecture de navigation à deux niveaux :

Bootstrap Navigation (Navigation globale)

```
sealed interface AppState {
    data object Loading : AppState          // Splash screen
    data object Unauthenticated : AppState  // Sign in
    data object Authenticated : AppState     // Main app
}
```

Authenticated Navigation (Navigation interne)

- **Bottom Navigation** avec 4 destinations :
  - Home (Actualités)
  - Agenda (Emploi du temps)
  - Grades (Notes)
  - Profile (Profil)

2.4 Injection de Dépendances

Utilisation de **Koin** pour l'injection de dépendances :

```
// Module Network
val networkModule = module {
    single { provideOkHttpClient() }
    single { provideRetrofit(get()) }
    single { get<Retrofit>().create(ApiService::class.java) }
}

// Module Repository
val repositoryModule = module {
    single<NewsRepository> { NewsRepositoryImpl(get()) }
    single<AgendaRepository> { AgendaRepositoryImpl(get()) }
    // ...
}

// Module ViewModel
val viewModelModule = module {
    viewModel { HomeViewModel(get()) }
    viewModel { AgendaViewModel(get()) }
    // ...
}
```

---

3. Fonctionnalités Implémentées

3.1 Authentification ✅

OAuth2 Flow

- Connexion via identifiants MyGES (email + mot de passe)
- Extraction du token depuis l'URL de redirection OAuth
- Stockage sécurisé du token avec Android Keystore
- Maintien de la session (auto-login)

Endpoints utilisés :

```
POST /oauth/authorize
Content-Type: application/x-www-form-urlencoded

username={email}&password={password}&grant_type=password
```

Réponse : Redirection vers URL contenant le token

```
kordis://oauth?access_token={TOKEN}
```

3.2 Écran d'Accueil (Home) ✅

Fonctionnalités :

- Carousel de bannières (images défilantes avec indicateurs)
- Liste d'actualités avec images
- Pull-to-refresh
- Gestion des états (Loading, Success, Error, Empty)

API Endpoints :

```
GET /me/news              // Liste des actualités
GET /me/news/banners      // Images du carousel
```

Composants UI :

- `HorizontalPager` pour le carousel
- `LazyColumn` pour la liste d'actualités
- `AsyncImage` (Coil) pour le chargement d'images
- Indicateurs de page personnalisés

3.3 Agenda (Emploi du temps) ✅

Fonctionnalités :

- Affichage des cours avec horaires
- Filtrage par date
- Informations complètes : salle, professeur, discipline
- Badge de type de cours (TD, CM, etc.)

API Endpoint :

```
GET /me/agenda?start={timestamp}
```

UI Features :

- Cards Material Design 3
- Groupement par date
- Affichage des heures de début/fin
- Indicateur visuel du type de cours

3.4 Notes (Grades) ✅

Fonctionnalités :

- Consultation des notes par année académique

- Filtrage par trimestre (T1, T2, T3)
- Affichage des moyennes
- Détails des examens (cours, date, note)

API Endpoint :

```
GET /me/{year}/grades     // year = 2024, 2025, etc.
```

Structure des données :

```
data class Grade(
    val trimester: String,      // "T1", "T2", "T3"
    val course: String,         // Nom du cours
    val grade: Double?,         // Note obtenue
    val average: Double?,       // Moyenne de la classe
    val date: String           // Date de l'examen
)
```

3.5 Profil Étudiant ✅

Informations affichées :

- **Informations personnelles** :
  - Nom, prénom
  - Email professionnel et personnel
  - Téléphone mobile et fixe
  - Adresse postale
- **Informations académiques** :
  - Programme d'études
  - Année académique
  - Statut étudiant
- **Contact d'urgence** :
  - Nom du contact
  - Téléphone d'urgence

API Endpoint :

```
GET /me/profile
```

3.6 Splash Screen ✅

- Vérification du token au démarrage
- Redirection automatique vers Home si authentifié

- Redirection vers Sign In si non authentifié
- Logo de l'application

---

4. Technologies et Dépendances

4.1 Stack Technique

Catégorie	Technologie	Version
UI	Jetpack Compose	BOM 2025.11.01
UI	Material Design 3	-
Architecture	MVVM + Clean Architecture	-
DI	Koin	4.1.1
Networking	Retrofit	3.0.0
HTTP Client	OkHttp	5.3.2
JSON	Moshi	1.15.2
Images	Coil	3.0.4
Logging	Timber	5.0.1
Navigation	Navigation Compose	2.9.6
Serialization	Kotlinx Serialization	-

4.2 Configuration SDK

- **Min SDK** : 28 (Android 9.0)
- **Target SDK** : 36 (Android 15)
- **Compile SDK** : 36
- **JVM** : Java 17
- **Kotlin** : 2.2.21
- **AGP** : 8.13.1

---

5. Interface Utilisateur

5.1 Design System

Material Design 3

L'application utilise Material Design 3 avec support de :

- **Dynamic Color** (Android 12+) : couleurs adaptées au fond d'écran
- **Dark Mode** : thème sombre automatique
- **Edge-to-Edge** : affichage plein écran avec gestion des insets

Système de couleurs

Thème clair et sombre avec Dynamic Color (Android 12+)

Système d'espacement

```
data class Spacing(
    val extraSmall: Dp = 4.dp,
    val small: Dp = 8.dp,
    val medium: Dp = 16.dp,
    val large: Dp = 24.dp,
    val extraLarge: Dp = 32.dp
)
```

5.2 Pattern Smart/Dumb Components

Smart Components (Screens) :

- Gèrent l'état via ViewModel
- Orchestrent la navigation

Dumb Components (Content) :

- Purement présentationnels
- Réutilisables et testables

5.3 Composants Réutilisables

- **BasicButton** : Bouton personnalisé avec état loading
- **LoadingScreen** : Écran de chargement
- **ErrorScreen** : Gestion d'erreur avec retry
- **EmptyScreen** : État vide

5.4 Navigation Bottom Bar

Barre de navigation Material 3 avec 4 destinations : Home, Agenda, Grades, Profile

---

6. Gestion des Données

6.1 Architecture Réseau

API Endpoints

- `POST /oauth/authorize` - Authentification
- `GET /me/profile` - Profil utilisateur
- `GET /me/agenda?start={timestamp}` - Emploi du temps
- `GET /me/news` - Actualités
- `GET /me/news/banners` - Bannières
- `GET /me/{year}/grades` - Notes

Configuration OkHttp

- Timeout : 20 secondes (connexion + lecture)
- AuthInterceptor : Injection auto du token Bearer
- LoggingInterceptor : Logs HTTP (debug uniquement)

6.2 DTOs et Mapping

Pattern **DTO** → **Entity** pour séparer Data Layer et Domain Layer.

Exemple : `NewsDto` (JSON) → `News` (modèle métier) via fonction d'extension `.toEntity()`

6.3 Gestion de l'État

Sealed Classes pour UI State

```
sealed interface HomeUiState {
    data object Loading : HomeUiState
    data class Success(
        val news: List<News>,
        val banners: List<Banner>
    ) : HomeUiState
    data class Error(val message: String) : HomeUiState
}
```

StateFlow dans ViewModel

```
class HomeViewModel(
    private val repository: NewsRepository
) : ViewModel() {
    private val _uiState = MutableStateFlow<HomeUiState>(Loading)
    val uiState: StateFlow<HomeUiState> = _uiState.asStateFlow()

    init { loadData() }

    private fun loadData() {
        viewModelScope.launch {
```

```
            try {
                val data = repository.getNews()
                _uiState.value = Success(data)
            } catch (e: Exception) {
                _uiState.value = Error(e.message ?: "Error")
            }
        }
    }
}
```

6.4 Stockage Local

SharedPreferences chiffrées via `LocalSettingsRepository` pour stocker le token OAuth2 de manière sécurisée (AES-GCM 256 bits + Android Keystore).

Limitation actuelle : L'application ne dispose pas de cache local (Room).

Conséquences :

- Toutes les données sont récupérées via l'API à chaque ouverture
- Pas de mode hors ligne disponible
- Dépendance totale à la connexion réseau

---

7. Sécurité

7.1 Chiffrement du Token

Le token OAuth2 est stocké de manière sécurisée via `LocalSettingsRepositoryImpl` avec les technologies suivantes :

Technologies utilisées :

- Android Keystore System pour stockage matériel sécurisé
- Algorithme AES-GCM 256 bits avec tag d'intégrité 128 bits
- Vecteur d'initialisation (IV) aléatoire de 12 bytes
- Stockage combiné IV + données chiffrées en Base64

Flux de chiffrement :

Le processus se déroule en 5 étapes :

1. Génération ou récupération de la clé AES-256 depuis le Keystore
2. Initialisation du cipher en mode chiffrement
3. Chiffrement du token avec AES-GCM
4. Combinaison de l'IV et des données chiffrées

5. Encodage Base64 et stockage dans SharedPreferences

Flux de déchiffrement :

Le processus inverse se déroule en 4 étapes :

1. Récupération des données Base64 depuis SharedPreferences
2. Extraction de l'IV (12 premiers bytes) et des données chiffrées
3. Initialisation du cipher en mode déchiffrement
4. Déchiffrement et retour du token en clair

Garantie de sécurité : La clé AES-256 ne quitte jamais le hardware Keystore, même en cas de root ou de dump mémoire.

7.2 Bonnes Pratiques de Sécurité

Implémenté :

- Token chiffré avec AES-GCM 256 bits
- Pas de credentials hardcodés dans le code
- HTTPS uniquement pour toutes les requêtes
- Logs sensibles désactivés en production

À améliorer :

- SSL Pinning pour protection contre attaques MITM
- Authentification biométrique (empreinte/face)
- Refresh token automatique avec expiration
- Obfuscation du code avec ProGuard/R8

---

8. Internationalisation

8.1 Support Multilingue

L'application supporte 2 langues :

- Anglais (par défaut)
- Français

8.2 Structure des Ressources

Les fichiers de traduction sont organisés ainsi :

- `res/values/strings.xml` : Anglais (défaut)
- `res/values-fr/strings.xml` : Français

- `res/values/colors.xml` : Couleurs
- `res/values/themes.xml` : Thèmes

8.3 Exemples de Traductions

Navigation :

Clé	Anglais	Français
home_title	Home	Accueil
agenda_title	Schedule	Agenda
grades_title	Grades	Notes
profile_title	Profile	Profil

Authentification :

Clé	Anglais	Français
signin_title	Sign In	Connexion
signin_button	Sign In	Se connecter
signin_error	Invalid credentials	Identifiants incorrects

8.4 Utilisation dans le Code

```
@Composable
fun SignInScreen() {
    Text(text = stringResource(R.string.signin_title))

    OutlinedTextField(
        label = { Text(stringResource(R.string.signin_email_label)) }
    )

    Button(onClick = { /* ... */ }) {
        Text(stringResource(R.string.signin_button))
    }
}
```

8.5 Détection Automatique

La langue de l'application s'adapte automatiquement aux paramètres système de l'appareil :

```
val locale =
ConfigurationCompat.getLocales(Resources.getSystem().configuration)[0]
```

```
// Si locale = "fr" → strings-fr.xml
// Sinon → strings.xml (English)
```

---

9. Points d'Amélioration

9.1 Fonctionnalités Manquantes

Base de Données Locale (Room) ❌

Impact : Absence de mode hors ligne
Solution : Implémenter Room avec stratégie network-first + cache fallback

Firebase Integration ❌

Services manquants :

- Analytics : Tracking utilisateur
- Crashlytics : Rapports de crash
- Cloud Messaging : Notifications push
- Remote Config : Configuration dynamique

Tests ❌

- Tests unitaires (JUnit, MockK)
- Tests d'intégration (Repository, ViewModel)
- Tests UI (Compose Test)

9.2 Améliorations UX

- **Pull-to-Refresh** : Généraliser sur tous les écrans
- **Pagination** : Implémenter Paging 3 pour les listes longues
- **Skeleton Loading** : Remplacer les spinners par des placeholders animés

9.3 Performances

- **Image Caching** : Optimiser les politiques de cache Coil (mémoire/disque)
- **LaunchedEffect** : Éviter les re-compositions inutiles avec des clés appropriées

9.4 Sécurité

- **SSL Pinning** : Protection contre les attaques MITM
- **Obfuscation** : Optimiser les règles ProGuard/R8
- **Biométrie** : Authentification par empreinte/face

9.5 Accessibilité

- **Content Descriptions** : Compléter sur tous les composants
- **Semantic Properties** : Améliorer la navigation TalkBack

---

10. Conclusion

10.1 Objectifs Atteints

✅ Obligatoire - Implémenté

- [x] ~~UI/UX Jetpack Compose~~
- [x] ~~Intégration API MyGES (https://github.com/Melv1no/myges-api)~~
- [x] ~~Internationalisation (EN/FR)~~
- [x] ~~Gestion optimale des ressources (pas de hard coding)~~
- [x] ~~Logo explicite~~

⚠️ Optionnel - Partiellement implémenté

- [x] ~~Authentification (OAuth2, pas Firebase)~~
- [ ] Base de données locale (Room) - **Non implémenté**
- [ ] Firebase - **Non implémenté**
- [ ] Service de cartographie - **Non implémenté**

10.2 Points Forts

1. **Architecture Solide**
   - Clean Architecture avec séparation claire des couches
   - MVVM avec gestion d'état réactive (StateFlow)
   - Injection de dépendances avec Koin
   - Navigation type-safe avec Kotlin Serialization
2. **UI Moderne**
   - Jetpack Compose 100%
   - Material Design 3 avec Dynamic Color
   - Dark Mode natif
   - Edge-to-edge display
3. **Sécurité**
   - Chiffrement du token avec Android Keystore
   - AES-GCM 256 bits
   - Pas de données sensibles en clair
4. **Qualité du Code**

- Kotlin idiomatique
- Coroutines pour l'asynchrone
- Logs structurés avec Timber
- Pattern Smart/Dumb components

5. **Internationalisation**
    - Support complet EN/FR
    - Toutes les chaînes externalisées
    - Adaptation automatique à la locale système

10.3 Points Faibles

1. **Pas de Mode Hors Ligne**
    - Absence de base de données locale (Room)
    - Dépendance totale au réseau
    - Pas de cache de données
2. **Tests Absents**
    - Aucun test unitaire
    - Aucun test d'intégration
    - Aucun test UI
3. **Monitoring Limité**
    - Pas de Firebase Analytics
    - Pas de Crashlytics
    - Logs uniquement en debug
4. **Performances**
    - Pas de pagination
    - Chargement complet des listes
    - Pas de préchargement d'images
5. **Fonctionnalités Limitées**
    - Pas de notifications push
    - Pas de partage de cours
    - Pas de chat/forums
    - Pas de cartographie

10.4 Roadmap Future

Version 1.1 - Stabilité

- [ ] Ajout de Room pour cache local
- [ ] Mode hors ligne complet
- [ ] Tests unitaires (couverture 80%)
- [ ] Firebase Crashlytics

Version 1.2 - UX

- [ ] Pagination des listes
- [ ] Skeleton loaders
- [ ] Animations de transition
- [ ] Haptic feedback

Version 1.3 - Features

- [ ] Notifications push (nouveaux cours, notes)
- [ ] Widget emploi du temps
- [ ] Export PDF des notes
- [ ] Partage de fichiers entre étudiants

10.5 Bilan

L'application **MyGES** constitue une **base solide** pour un projet étudiant :

- Architecture professionnelle et maintenable
- Code moderne utilisant les dernières technologies Android
- Interface utilisateur soignée et intuitive
- Intégration API fonctionnelle

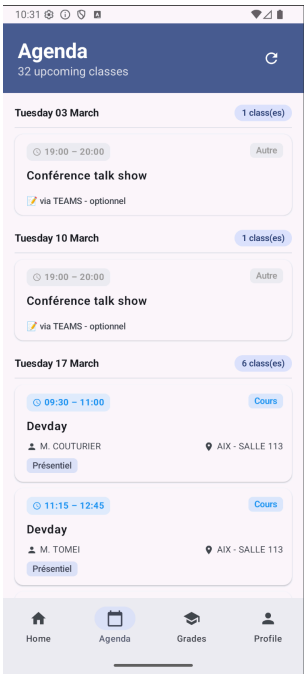
Les **limitations actuelles** (absence de Room, tests, Firebase) sont des **améliorations futures** identifiées et peuvent être ajoutées progressivement sans refonte majeure grâce à l'architecture modulaire mise en place.

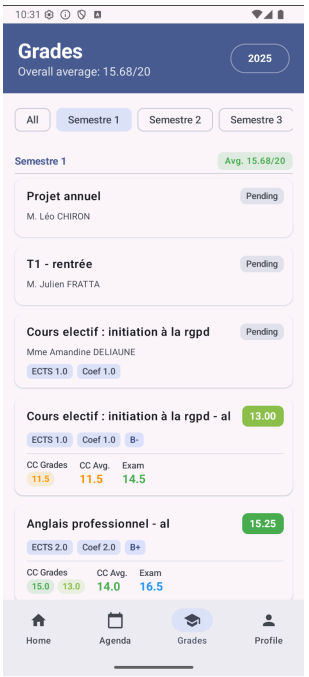
Le projet **répond aux exigences obligatoires du cahier des charges** et pose les fondations d'une application extensible et évolutive.

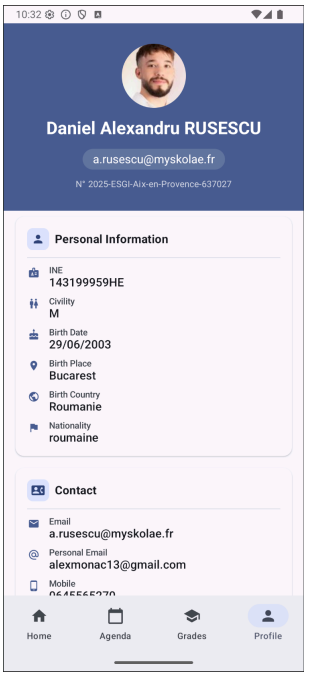
Annexes

A. Captures d'écran de l'application

Cette section est dédiée aux captures d'écran de l'application MyGES, illustrant les différents écrans et fonctionnalités implémentés.







Fin du rapport technique